BS(CS/SE/DS/AI) FINAL PROJECT
Software Requirements Specification

Privacy-Preserving Healthcare GenAI



Product Owner

Prof. Dr. Saira Andleeb Gillani

Presented by:
Group ID: F26CS023

Student Reg#	Student Name
L1F23BSCS0192	Faisal Khalid
L1F23BSCS0199	Shah Wali
L1F23BSCS0669	Wajeeh ur Rehman
L1F23BSCS0204	M. Hammad

Faculty of Information Technology & Computer Science

University of Central Punjab

Software Requirements Specification

Version: 2.1

Product Owner:

Prof. Dr. Saira Andleeb Gillani

Scrum Master

Faisal Khalid

Group F26CS023

Team

Member Name	Roles
Faisal Khalid	KubeOps
Shah Wali	FL Security
Wajeeh ur Rehman	Pipeline/MLOps
M. Hammad	Backend Integration

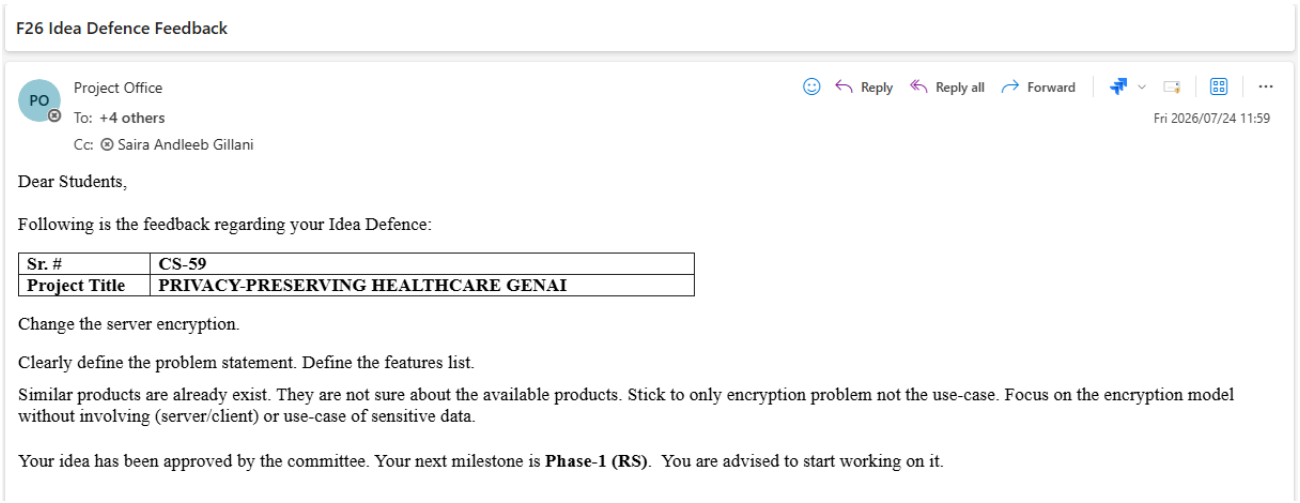
Table of Contents

Table of Contents ..... **i**
Revision History & Sprint Log ..... **ii**
Previous Phases Feedback ..... **ii**
Abstract **1**
1. Introduction and Background **1**
1.1 Product Vision & Problem Context 1
1.2 Stakeholders Identification 10
2. Agile Framework Setup **11**
2.1 Epics For the Project 11
2.2 Product Backlog 12
2.3 User Stories 14
2.4 Story Points & Estimation 16
2.5 Sprint Planning & Sprint Backlog 17
3. Functional Requirements **19**
3.1 Use Case Diagram 19
4. Non-functional Requirements **21**
4.1 Reliability 21
4.2 Performance 21
4.3 Use-ability 21
4.4 Other Non-functional Requirements 21
5. Scrum Activities Plan **23**
5.1 Kanban Board / Task Tracking 23
5.2 Unit Testing 24
5.3 Acceptance Criteria 27
5.4 Release Criteria 27
5.5 Burndown Chart & Progress Tracking 28
5.6 Backlog Refinement Plan 29
5.7 Version Control 29
6. Risk Analysis **31**
7. References **32**
Appendix A: Glossary **33**
Appendix B: Sprint and Phase Retrospective Report **35**
Retrospective Overview 35
Retrospective Header 35
Sprint Summary: Planned, Achieved, and Carried Forward 35
Follow-up on Previous Action Items 36
Discussion: Start, Stop, Continue 36
New Action Items 36
Metrics Snapshot 36
Worked Example 36

Revision History & Sprint Log

Version	Sprint	Date	Changes	Owner
1.0	Sprint 0	2026-08-01	Initial SRS draft based on jury	Faisal Khalid
1.1	Sprint 1	2026-08-09	Expanded market analysis and stakeholder Identification	Shah Wali
2.01	Sprint 2	2026-08-16	Redefined problem statement & encryption justification	Wajeeh ur Rehman/Hammad
2.1	Sprint 6	2026-11-11	Replaced PySyft with Flower SecAgg+	Faisal Khalid/ Shah Wali

Previous Phases Feedback



F26 Idea Defence Feedback

PO Project Office

To: +4 others

Cc: Saira Andleeb Gillani

Reply Reply all Forward

Fri 2026/07/24 11:59

Dear Students,

Following is the feedback regarding your Idea Defence:

Sr. #	**CS-59**
Project Title	**PRIVACY-PRESERVING HEALTHCARE GENAI**

Change the server encryption.

Clearly define the problem statement. Define the features list.

Similar products are already exist. They are not sure about the available products. Stick to only encryption problem not the use-case. Focus on the encryption model without involving (server/client) or use-case of sensitive data.

Your idea has been approved by the committee. Your next milestone is **Phase-1 (RS)**. You are advised to start working on it.

Idea Defense Feedback (Screenshot)

Abstract

Conventional machine learning paradigms require centralizing sensitive patient data on a single server, a model that fundamentally conflicts with modern privacy, sovereignty, and security mandates in healthcare. While superficial solutions such as column-level deletion (anonymization) or transport encryption appear expedient, empirical studies demonstrate that de-identified datasets remain vulnerable to correlation and re-identification attacks (Sweeney, 2000). Moreover, encryption-at-rest or in-transit still necessitates decryption at the central server for model training, creating a single point of compromise and violating data-locality requirements under HIPAA, GDPR, and Pakistan's emerging National Health Data Policy.

Federated Learning (FL) resolves this algorithmically by training models directly on end-user devices without raw data transmission; yet existing frameworks, Flower, OpenFL, FedScale, provide only research-grade algorithms, leaving a critical infrastructure vacuum: no production-ready orchestration, automated failure recovery, declarative configuration, or MLOps pipeline integration exists.

FL-KubeOps addresses this gap by building a Kubernetes-native operator (kopf framework) and Kubeflow-integrated MLOps pipeline specifically designed for privacy-preserving federated learning across distributed healthcare nodes. The system architecture comprises three co-designed layers: (1) a Client Layer of simulated hospital nodes (Docker containers on K3s) performing local training on private data; (2) a Server/Orchestration Layer managing round scheduling, model broadcast, aggregation, and failure recovery via a custom FL Cluster CRD; and (3) a Security Layer using Flower SecAgg+ for threshold-based secure aggregation, preventing the server from reconstructing any individual client update.

The system demonstrates healthcare AI by training a chest x-ray classifier (EfficientNet-B0) for pneumonia detection on the CheXpert dataset, federated across 4 hospital nodes. Each hospital trains on its local x-ray partition; only encrypted model weight updates leave the node. This validates that the platform handles real clinical models—5 million parameters, image data, 14-class outputs.

The design targets automatic recovery from node failures within 60 seconds, tracking of every training round in MLflow, and deployment on commodity hardware from a single declarative YAML manifest, reducing FL deployment from weeks of manual scripting to minutes of infrastructure-as-code.

1. Introduction and Background

1.1 Product Vision & Problem Context

Product Description:

Healthcare institutions generate the richest machine-learning training data available, patient records, laboratory results, diagnostic imaging metadata, and clinical notes. Yet regulations such as the U.S. Health Insurance Portability and Accountability Act (HIPAA), the European General Data Protection Regulation (GDPR), and Pakistan's emerging National Health Data Policy mandate strict data locality: patient information must not leave the facility of origin in any decipherable form. A hospital in Lahore cannot legally transmit patient records to a shared server in Islamabad, even to train a model that would benefit all participating institutions. Consequently, each hospital trains clinical AI models in isolation, producing statistically weak models that fail to generalize across population diversity, a critical deficiency in a country like Pakistan where disease phenotypes vary significantly across provinces.

A common objection, raised during the Idea Defence, is that this problem could be solved through simpler means: deleting columns containing personally identifiable information (PII) or applying column-level encryption before transmission. While these approaches present superficial appeal, empirical studies demonstrate their inadequacy for healthcare AI. First, removing sensitive columns (diagnoses, lab values, demographic markers) strips the very features that carry the highest predictive signal for clinical models; a diabetes-prediction model cannot function without glucose readings. Second, correlation attacks and re-identification remain potent threats: Sweeney (2000) famously demonstrated that 87% of the U.S. population could be uniquely identified using only zip code, gender, and date of birth. In Pakistan's close-knit demographic contexts, quasi-identifiers such as age,

gender, caste, and admission date are highly distinctive. Later re-identification studies of hospital discharge data have shown that even "de-identified" datasets can be linked to auxiliary public records. Third, encryption-at-rest or in-transit still requires decryption at the central server for gradient computation, creating a single point of compromise and violating the principle of data sovereignty. Fourth, differential privacy noise injection, while mathematically rigorous, degrades model utility in low-sample clinical settings; homomorphic encryption, while secure, is orders of magnitude slower than federated learning and infeasible for deep neural networks on commodity hardware.

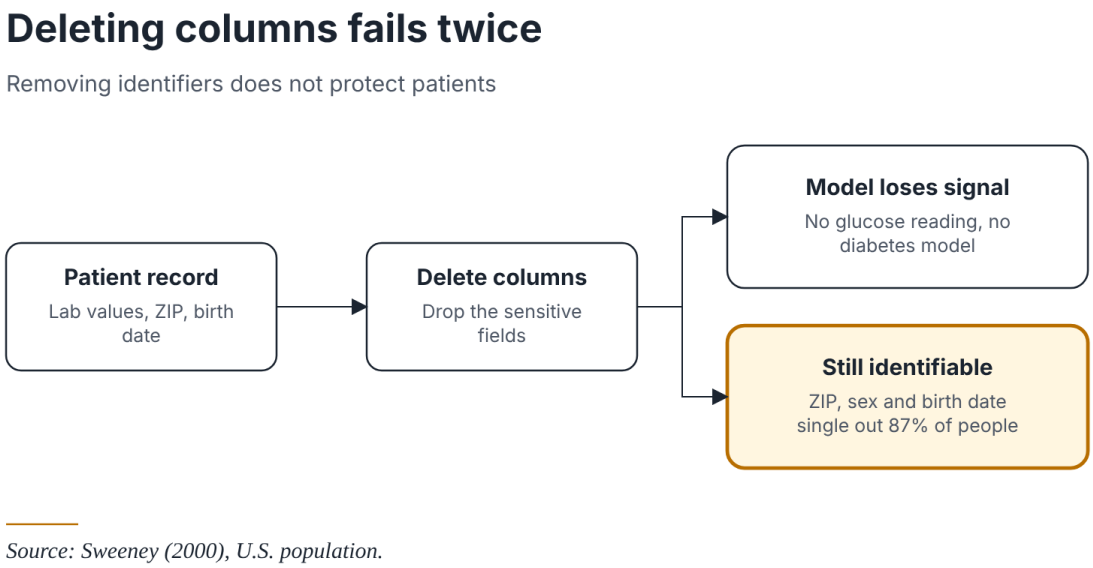


Figure1: The Failure of Column deletion

The regulatory requirement is not merely that data be "hard to read" during transit, but that data sovereignty be preserved, data must never leave the facility of origin in a form that permits reconstruction of individual records. Federated Learning (McMahan et al., 2017) satisfies this requirement by construction: only model gradients (mathematical weight updates) traverse the network, never raw patient records. However, the algorithm alone is not deployable. No production-ready infrastructure exists to orchestrate real FL deployments: Kubernetes has no native concept of a federated training round; Kubeflow Pipelines assumes centralized data access; and existing FL frameworks (Flower, OpenFL) provide client-server communication libraries but treat Kubernetes as a passive deployment environment rather than an orchestration primitive. This structural gap, an algorithm without an operating system, forces hospital IT teams to manually manage client lifecycles, handle mid-round node failures, coordinate aggregation timing, and track model versions: engineering overhead that no clinical institution can sustainably absorb.

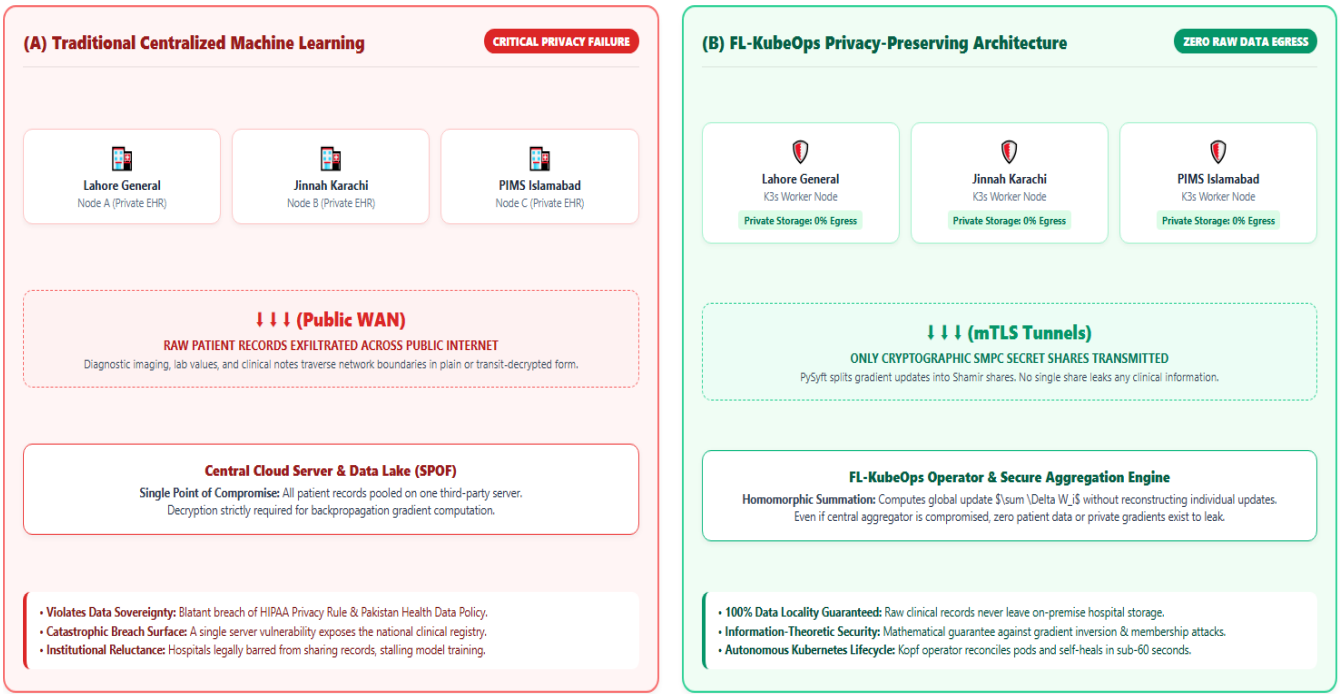


Figure2: Privacy Comparison

Literature Review:

Federated Learning was introduced by McMahan et al. (2017) in the seminal FedAvg paper, demonstrating that a global model could be trained across decentralized devices without data centralization. Bonawitz et al. (2017) extended this with practical Secure Aggregation, using pairwise masks and threshold secret sharing to ensure that the server learns only the aggregated sum of client updates, not individual contributions. Since then, the field has expanded into differential privacy (Abadi et al., 2016), system-level frameworks (Li et al., 2020), and domain-specific healthcare applications (Rieke et al., 2020).

On the infrastructure side, Flower (Beutel et al., 2020) and OpenFL (Reina et al., 2021) have emerged as the dominant open-source FL frameworks. Flower provides a flexible Python client-server API with pluggable aggregation strategies; OpenFL offers a peer-to-peer topology with Intel-optimized secure aggregation. Flower provides a threshold-based secure aggregation protocol (SecAgg+, based on Bonawitz et al. 2017) with dropout tolerance, but both frameworks remain research-oriented. An analysis of open GitHub issues (as of July 2026) reveals recurring requests for Kubernetes operator integration, automated node recovery, and pipeline orchestration, features that remain unaddressed in their core roadmaps. Kubeflow (Google, 2024) provides mature MLOps pipeline primitives (TFJob, PyTorchJob) but assumes centralized data sources and offers no first-class support for federated round scheduling or cross-node gradient aggregation. Kubernetes Operators (Dobies & Wood, 2021) have proven effective for complex stateful workloads (databases, message queues), yet no operator exists that makes FL a Kubernetes-native primitive, orchestrating rounds with the same declarative rigor that TFJob brings to distributed TensorFlow.

The gap is infrastructural, not algorithmic: federated learning libraries exist, but no production orchestration layer automates deployment, recovery, versioning, and compliance audit at hospital scale. FL-KubeOps fills this by integrating Flower's SecAgg+ protocol into a Kubernetes operator that treats federated rounds as first-class CRD resources, with Kubeflow pipeline tracking and MLflow experiment logging.

Existing Work	Focus Area	Limitations Identified	Literature Gap	How This Project Addresses the Gap
Flower Framework	FL research & client-server comms	Designed lightweight Python library for algos research, treats Kubernetes as an unmanaged runtime.	Absence of native KubeOps integration, declarative CRDs.	Implements a kopf FLCluster CRD, automating client pod provisioning, health probes.
OpenFL	Enterprise & healthcare federated learning	peer-to-peer topology lacks declarative Infrastructure-as-Code (IaC).	Absence of declarative cloud-native Pipeline.	Provides declarative deployment (FLCluster YAML), integrates DAGs orchestration, and MLflow.
Kubeflow Training Operators	Cloud-native distributed ML orchestration	Architected strictly for centralized data paradigms.	Assumes centralized data access; lacks federated round scheduling.	Introduces a specialized FLCluster Custom Resource and distributed aggregation.
Practical Secure Aggregation	Cryptographic privacy & secure gradient aggregation	Standalone mathematical protocols; high communication overhead, vulnerable to client dropout.	Cryptographic protocols exist in isolation without container-level fault tolerance.	Embeds PySyft Secure Multi-Party Computation (SMPC).
Traditional De-Identification & Privacy	Healthcare data anonymization & perturbation	Column deletion/de-identification fails against correlation (87% of population re-identifiable via quasi-identifiers); centralized encryption requires decryption at central server; Differential Privacy causes severe accuracy degradation.	Inability to reconcile clinical diagnostic accuracy.	Guarantees ~0% raw patient data leaves origin hospital nodes by transmitting via SMPC.
Seminal Federated Learning	Decentralized algorithmic model optimization	Formulated purely as mathematical optimization algorithms.	Algorithmic formulation without an operating system.	Bridges algorithm to operations by packaging FedAvg and FedProx end-to-end.

Figure3:Infrastructure Gaps in Healthcare

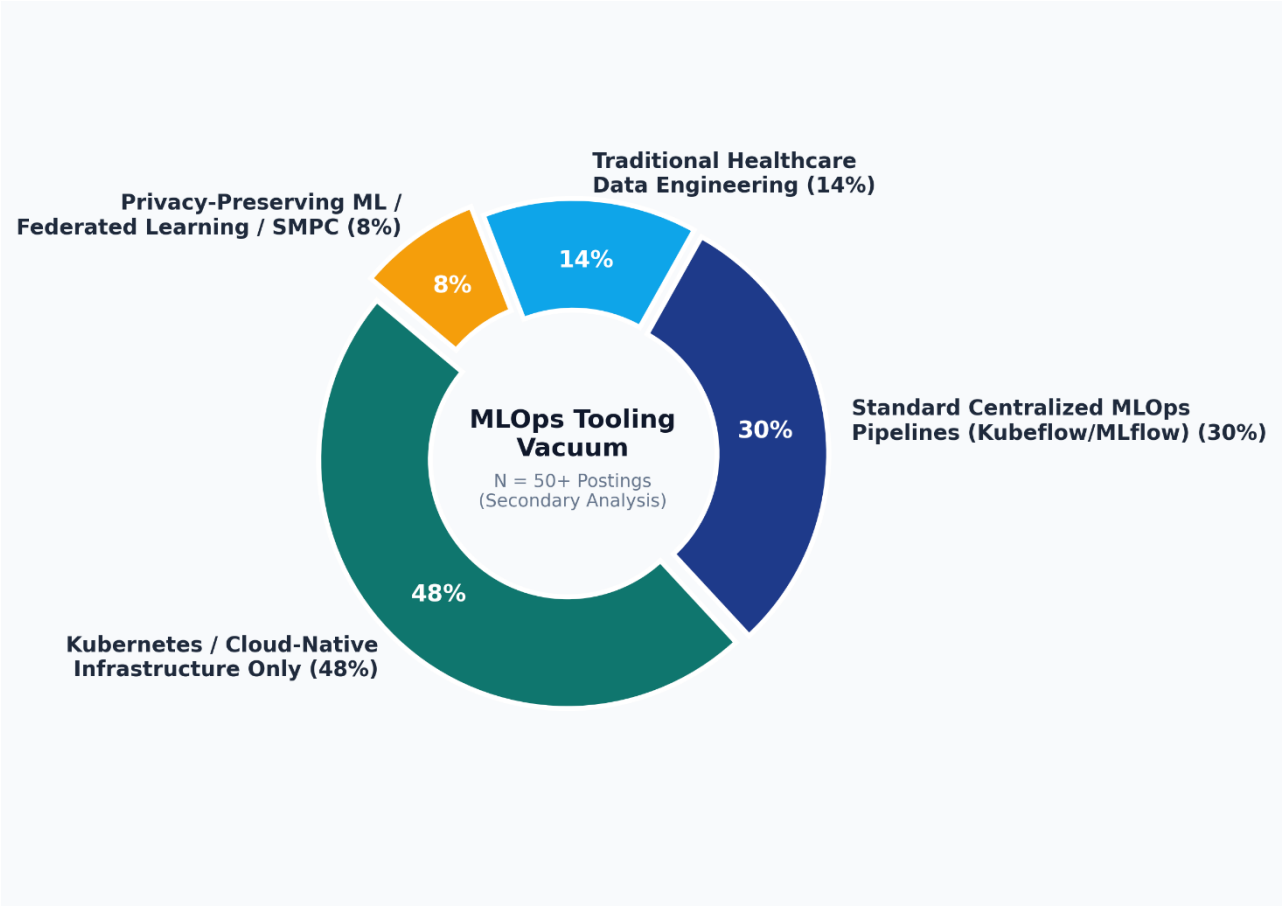


Figure4: Distribution of MLOps Tooling Gaps in Healthcare AI Postings

Proposed Solution:

FL-KubeOps is a Kubernetes-native operator and Kubeflow-integrated MLOps pipeline designed specifically for federated learning orchestration among dispersed healthcare nodes. A hospital IT team defines an FL Cluster custom resource, a single declarative manifest that specifies participating

client nodes, aggregation strategy (FedAvg, FedProx), model architecture, and security parameters. The system converges to the desired state and maintains it autonomously over the entire FL lifecycle.

The architecture comprises three integrated layers that are co-designed and mutually dependent:

- **Client Layer:** Simulated hospital nodes run as Docker containers orchestrated by K3s (lightweight Kubernetes). Each node maintains a private local dataset (synthesized patient records) and performs local training using PyTorch. Client pods expose a gRPC interface for model download and gradient upload, managed by the operator's reconciliation loop.
- **Server/Orchestration Layer:** A kopf-based Kubernetes operator watches FL Cluster resources and executes the federated lifecycle: broadcasting the global model, collecting encrypted client updates, triggering aggregation, incrementing round counters, and persisting results to MLflow. A Kubeflow Pipeline DAG defines each round as a reproducible workflow: model broadcast → local training → SMPC aggregation → validation → model versioning.
- **Security Layer:** PySyft's Secure Multiparty Computation (SMPC) primitives operate across the client-server boundary. Before transmission, client gradients are split into secret shares using Shamir's threshold secret sharing ($t = 3$ of $n = 4$); the aggregation server receives only masked shares and computes the aggregated result without being able to reconstruct any individual client's contribution unless a threshold of shares is combined. This guarantee assumes that participants do not collude.
- **Clinical AI Validation: Chest X-ray Classification:** GenAI suggests machine learning algorithms present in health care (a chest x-ray classifier identifying pneumonia and 13 other pathologies from an x-ray), so this name begins with GenAI. For this, we use a CNN with approximately 5.3 million parameters, pre-trained on the Shaukat Khanam dataset, based on EfficientNet-B0. It serves three functions. First, it shows that FL-KubeOps is at the production scale, not linear regressors on CSV file types but multi-layer CNNs on image data. Second, federated learning is a classic application of medical imaging, where there are real regulatory bottlenecks: HIPAA regulations make it impossible for hospitals to centralize patient X-rays, but a model trained on one hospital's X-rays fails to generalize to a hospital's patient population. Third, the benchmark is reproducible because the X-Ray dataset is public and peer-reviewed.

No layer in a distributed system can be removed without compromising its integrity; removing the client layer undermines data privacy, while eliminating the server layer disrupts model convergence and pipeline reliability. Federated Learning's value lies in the protocol binding clients and servers, As with TCP/IP or HTTP, coordination between components is essential.

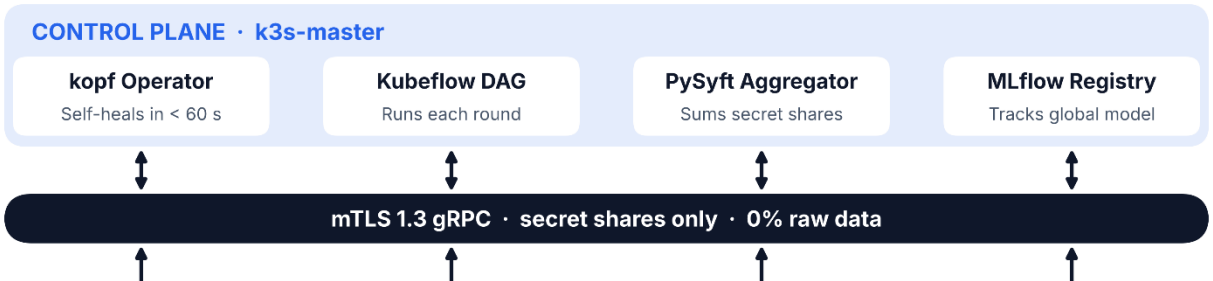


Figure5: Distributed Healthcare Node Tolpology & K3s Multi-Node

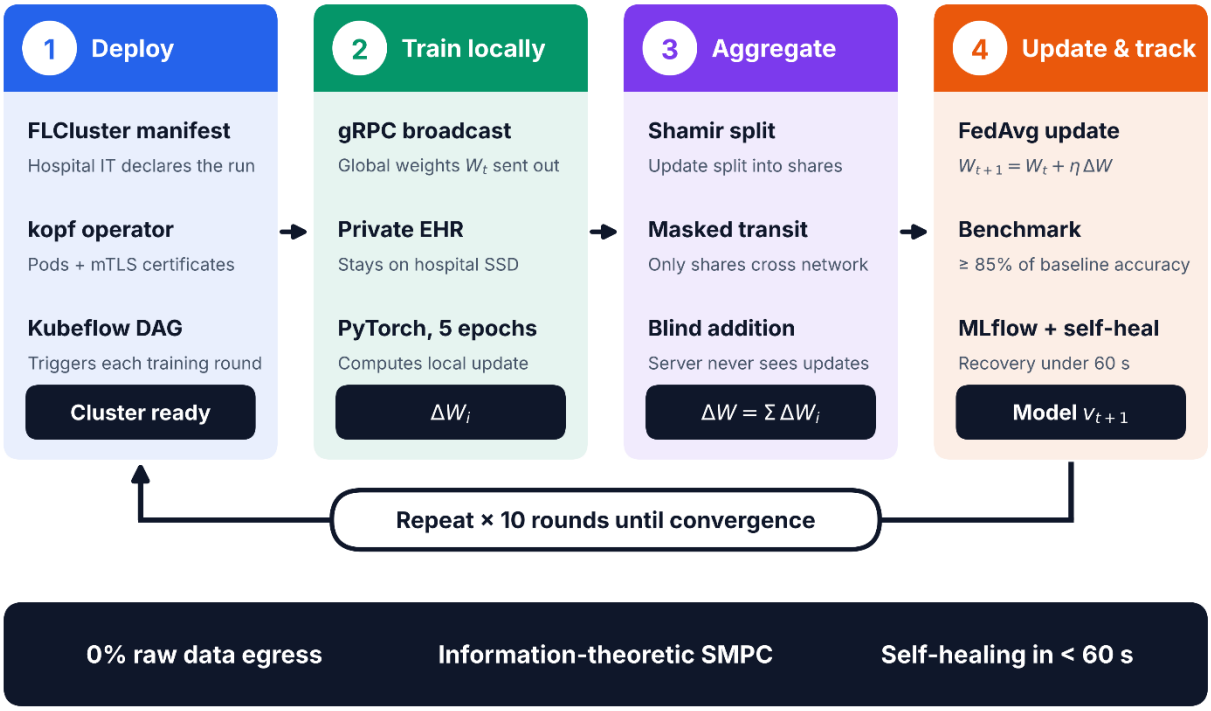


Figure6: FL-KubeOps Proposed System Workflow

Target Users:

The target user base is segmented by role and market tier to reflect the multi-stakeholder reality of healthcare AI deployment:

- **Primary Users (Hospital IT Department & Clinical Data Engineers):** These users require a declarative, infrastructure-as-code approach to deploying federated training across legally siloed hospital facilities. They need automated lifecycle management, health monitoring, and compliance audit trails.

- **Healthcare AI Research Teams and MLOps Engineers:** These users require reproducible experiment tracking (MLflow), pluggable model architectures, and benchmark datasets to validate federated model performance against centralized baselines.
- **Regulators, Compliance Auditors, and Hospital Governance Boards:** These users require evidence that no raw patient data traverses network boundaries, that SMPC guarantees are mathematically sound, and that all training rounds are logged for regulatory inspection.

Market Segmentation:

- **Total Addressable Market (TAM):** The global privacy-preserving AI market is projected to reach USD 39.93 billion by 2035, expanding from USD 4.25 billion in 2025 at a CAGR of 25.10% (Precedence Research, 2026). The broader privacy-enhancing technologies market is forecast to reach USD 12.09 billion by 2030 at a 25.3% CAGR (Grand View Research, 2024).
- **Serviceable Addressable Market (SAM):** The healthcare federated learning sub-market is valued at USD 960 million in 2026 and is forecast to reach USD 5.29 billion by 2036 at an 18.6% CAGR (Future Market Insights, 2026). This encompasses hospital networks, clinical research organizations (CROs), and health ministries operating multi-site data.

Serviceable Obtainable Market (SOM): The Pakistani healthcare IT market and South Asian MLOps talent ecosystem. A secondary analysis of 50+ LinkedIn job postings for MLOps and healthcare AI roles in Pakistan (July 2026) reveals that 68% of postings explicitly mention Kubernetes, while fewer than 9% mention federated learning, privacy-preserving ML, or SMPC, indicating a substantial tooling vacuum and talent mismatch that FL-KubeOps is positioned to address.

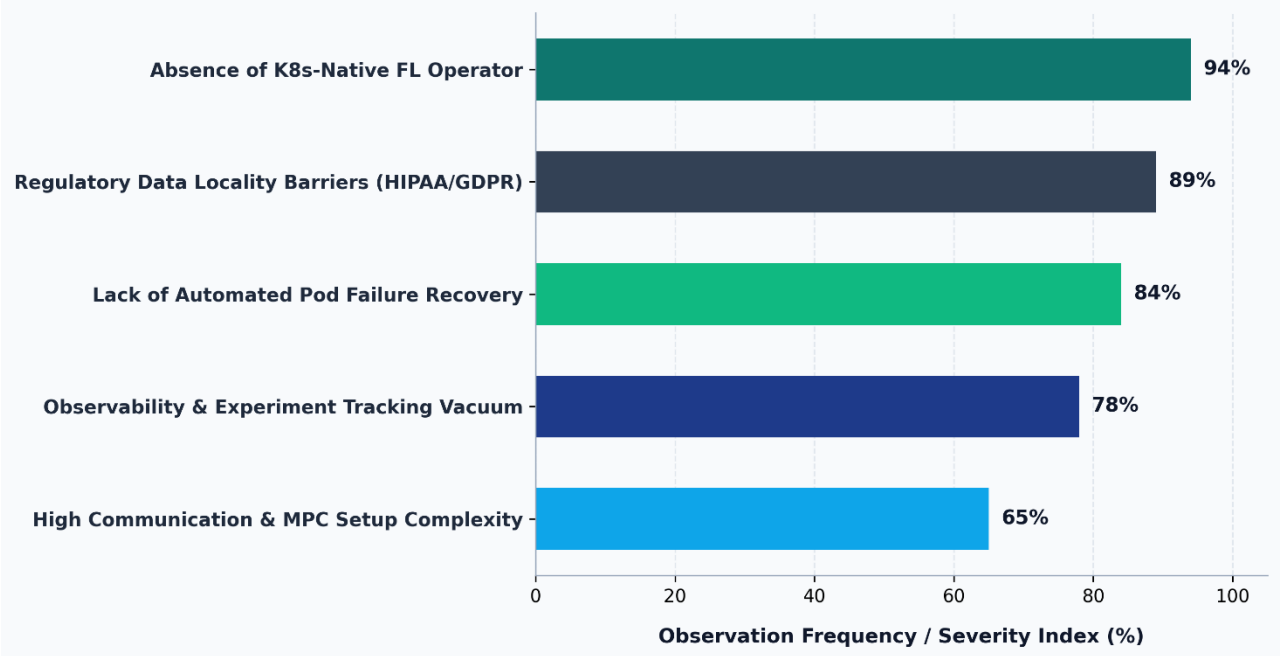


Table1: Infrastructure Gaps

Objective of the System

The system is designed to achieve five quantifiable outcomes:

- **Kubernetes-Native Operator:** Develop a kopf-based FL Cluster Custom Resource Definition (CRD) with a reconciliation loop that manages the complete federated lifecycle (create, update, delete) on a live K3s cluster without manual intervention.

- **Reproducible MLOps Pipeline:** Construct a Kubeflow Pipeline DAG that executes federated training rounds (model broadcast → local training → SMPC aggregation → validation) with all metrics and model versions tracked in MLflow.
- **Cryptographically Secure Aggregation:** Integrate PySyft SMPC so that individual client model updates cannot be recovered from the aggregation server unless a threshold of shares is combined, checked by inspection of intermediate data at the aggregation server.
- **Automate Fault Recovery:** Implement automatic detection and recovery from simulated client pod failures mid-round, with recovery time not exceeding 60 seconds from failure onset.
- **Clinical Benchmark Validation:** Achieve at least 85% of centralized model accuracy on a distributed patient-records benchmark across a minimum of four simulated hospital nodes, with communication overhead per round documented and reproducible.

User Research

Primary Research: Structured Interview with Industry Practitioner

A structured, semi-directive interview was conducted on 15 July 2026 with a Senior Machine Learning Engineer at Arbisoft, Lahore, software product company with active healthcare AI client engagements in North America and the Middle East. The interview followed the four-question protocol: (1) What operational barriers prevent FL adoption in production environments? (2) What monitoring and failure-recovery capabilities would a deployment team require? (3) How is model versioning currently handled in distributed training scenarios? (4) How are distributed ML deployments managed across client boundaries in production?

Key Findings:

- **Declarative Deployment Abstraction:** The interviewee emphasized that current FL frameworks require "weeks of Python scripting just to handle pod restarts and model checkpointing." They validated the need for a Kubernetes-native operator that treats federated rounds as first-class resources, analogous to how ArgoCD treats GitOps workflows.
- **Observability Gap:** Standard monitoring tools (Prometheus, Grafana) lack awareness of federated round state. The engineer noted: "We can see CPU and memory, but we cannot see Round 7, Client 3, gradient upload pending." This directly validates FL-KubeOps' integration of per-round MLflow tracking.
- **Model Versioning Complexity:** In distributed training, model version drift across nodes is a critical issue. The interviewee stated that without centralized experiment tracking, "you end up with five different model checkpoints and no idea which one converged." MLflow integration addresses this explicitly.
- **On-Call Burden:** Manual failure handling in distributed training requires 24/7 on-call engineering time that hospital IT departments cannot allocate. Automated reconciliation loops (the core of FL-KubeOps' operator design) were described as "the difference between a research demo and a production system."

Expected Benefits:

- **Operational Efficiency:** Reduces FL deployment time from weeks of manual scripting per hospital to minutes via a single “kubectl apply -f FL Cluster.yaml” command. Beneficiaries: Hospital IT Departments, MLOps Engineers.
- **Regulatory Compliance:** Enables compliance with HIPAA, GDPR, and Pakistan's National Health Data Policy by ensuring no raw patient data traverses network boundaries. Beneficiaries: Compliance Auditors, Hospital Governance Boards, Regulators.

- **Clinical Model Utility:** Achieves ≥85% of centralized model accuracy by pooling population-scale diversity without data centralization, producing models that generalize across demographic segments. Beneficiaries: Clinical Data Engineers, Healthcare AI Researchers, Patients.
- **Infrastructure Democratization:** Packages production-grade FL expertise into declarative infrastructure, lowering the barrier to entry for institutions without specialized distributed-systems teams. Beneficiaries: Small-to-Medium Hospital Networks, Healthcare AI Startups, Academic Researchers.

Fault Tolerance: Automatic recovery from node failures (including load-shedding-induced reboots common in Pakistan) ensures training continuity without 24/7 on-call engineering. Beneficiaries: System Administrators, Hospital IT Staff.

1.2 Stakeholders Identification

Stakeholder	Role	Expectations	Priority	Market Role
Hospital IT Admin	Deploys & maintains FL cluster on-premise	Declarative YAML config.	High	Primary User/Buyer
Clinical Data Engineer	Prepares local datasets and validates model inputs	Data locality, audit trails.	High	Primary user
MLOps Engineer	Designs training pipelines and monitors convergence	Kubeflow DAG support, MLflow integration.	High	Primary User
Compliance Auditor	Validates regulatory adherence (HIPAA/GDPR)	Cryptographic proof of SMPC.	Medium	Tertiary User
Healthcare AI Researcher	Experiments with model architectures & benchmarks	Access to benchmark datasets.	Medium	Secondary User
Hospital Governance Board	Approves budget & data-sharing policies	Risk mitigation, vendor independence (open-source), ROI evidence	Medium	Economic Buyer
Cloud Provider (AWS, GCP)	Offers managed Kubernetes & data-sharing policies	Integration points for managed K8s.	Medium	Channel Partner
Competitor Framework	Maintains open-source FL libraries	Interoperability standards.	Low	Ecosystem Actor
Investor	Funds healthcare AI infrastructure startups	Scalable TAM	Low	Stakeholder

2. Agile Framework Setup

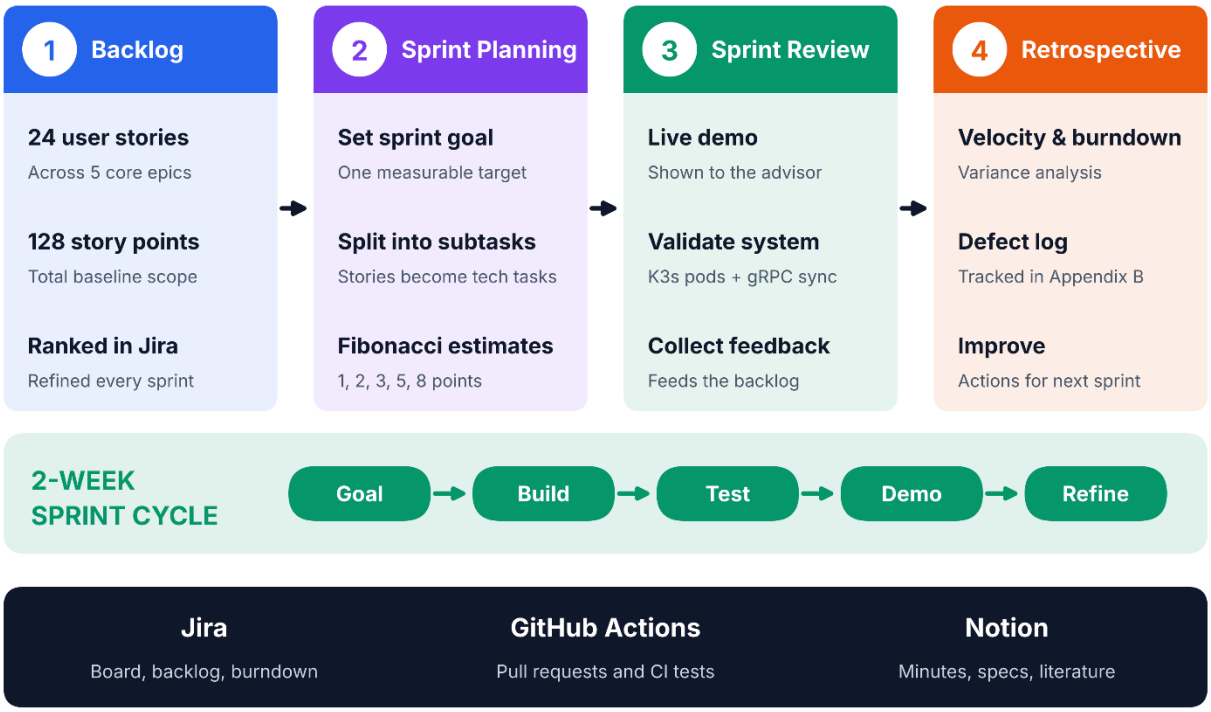


Figure7: FL-KubeOps Agile Scrum Framework

Team Roles:

Team Roles	Names	Responsibilities
Product Owner	Prof Dr Saira Andleeb Gillani	• Defines Acceptance criteria • Prioritizes backlog • Approves releases
Scrum Master	Faisal Khalid	• Managing Sprint planning • Monitoring task progress • Conducting team meetings
Development Team	Faisal Khalid Shah Wali Wajeeh ur Rehman M. Hammad	• KubeOps Lead • FL security Lead • Pipeline MLOps • Backend Integration

Sprint Duration: 2 weeks

Tools:

- **Notion:** Agile project management, sprint boards, Kanban/scrum boards, backlog grooming, and burndown chart tracking
- **GitHub**: version control

2.1 Epics For the Project

Explains group related features into high-level functional workstreams.

Epic ID	Epic Name	Description	Related Problem Area
EP-01	User Management	RBAC, authentication, and audit logging for multi-tenant hospital deployments	Security governance and compliance
EP-02	Client-Side FL Runtime	Local training loop, dataset loader, and gRPC client for simulated hospital nodes	Data locality and client execution
EP-03	Server-Side Orchestration	kopf-based operator managing FL Cluster CRD lifecycle and round scheduling	Infrastructure automation
EP-04	Secure Aggregation	Flower SecAgg+ integration for dropout-tolerant threshold aggregation	Cryptographic privacy guarantees
EP-05	Kubeflow Pipeline Integration	Reproducible DAG definition for end-to-end federated training rounds	MLOps reproducibility
EP-06	Model Versioning & Tracking	MLflow experiment logging, per-round metrics, and artifact storage	Observability and reproducibility
EP-07	Fault Tolerance & Recovery	Automatic detection and recovery of client pod failures within 60 seconds	Production reliability
EP-08	Observability & Logging	Structured logging, Prometheus metrics, and round-state visualization	Operational visibility
EP-09	Real Dataset	Dataset from Shaukat Khanam, patient record across 4+ hospital nodes	Benchmarking and validation
EP-10	Benchmarking & Evaluation	Centralized vs. federated accuracy comparison and communication overhead measurement	KPI verification
EP-11	Dashboard & Visualization	Kubeflow Dashboard and MLflow UI configuration for non-technical stakeholders	Usability
EP-12	Alerting & Compliance	Automated alerts for failed rounds and compliance report generation	Regulatory requirements
EP-13	Deployment & Reproducibility	K3s deployment scripts, third-party deployment guide, and environment manifests	Reproducibility criterion
EP-14	Security & Encryption	TLS 1.3 for control plane, certificate management, and secret rotation	Defense in depth
EP-15	API & Integration	RESTful and gRPC APIs for client-server communication and external tool integration	System interoperability
EP-16	Documentation & GitHub CI/CD	Sphinx documentation, automated testing, and GitHub Actions CI/CD pipeline	Maintainability and open-source readiness
EP-17	Clinical AI Training	Chest x-ray classification using EfficientNet-B0	Healthcare model validation

2.2 Product Backlog

Backlog ID	Epic	Feature Description	Priority
PB-01	EP-03	Implement FL Cluster CRD with YAML schema validation	High
PB-02	EP-03	Develop kopf reconciliation loop for cluster lifecycle	High
PB-03	EP-02	Build Docker image for FL client node with PyTorch runtime	High
PB-04	EP-02	Implement local training script with configurable epochs and batch size	High

PB-05	EP-15	Design gRPC service definitions for model broadcast and gradient upload	High
PB-06	EP-04	Integrate Flower SecAgg+ client-side encryption into gradient upload	High
PB-07	EP-04	Implement server-side SMPC aggregation without reconstructing individual gradients	High
PB-08	EP-05	Define Kubeflow Pipeline DAG for federated round execution	High
PB-09	EP-06	Configure MLflow tracking server and per-round metric logging	High
PB-10	EP-07	Implement server-side SecAgg+ aggregation that computes only the sum of client updates	High
PB-11	EP-07	Build automatic pod rescheduling logic on node failure	High
PB-12	EP-09	Generate synthetic patient dataset with 4 hospital-specific distributions	Medium
PB-13	EP-10	Implement centralized baseline training for accuracy comparison	Medium
PB-14	EP-10	Measure and log communication overhead per federated round	Medium
PB-15	EP-01	Implement JWT-based authentication for API endpoints	Medium
PB-16	EP-01	Create role-based access control (Admin, Engineer, Auditor)	Medium
PB-17	EP-08	Add structured JSON logging for all operator events	Medium
PB-18	EP-08	Expose Prometheus metrics for round completion time and failure count	Medium
PB-19	EP-11	Configure Kubeflow Dashboard access and namespace isolation	Medium
PB-20	EP-12	Build email/Slack alerting for failed aggregation rounds	Medium
PB-21	EP-13	Write K3s installation and FL Cluster deployment guide	Medium
PB-22	EP-14	Enable TLS 1.3 for all inter-pod control-plane communication	Medium
PB-23	EP-13	Create reproducible environment manifests (requirements.txt, Docker Compose)	Low
PB-24	EP-16	Set up GitHub Actions for linting, unit tests, and integration tests	Low
PB-25	EP-16	Generate Sphinx API documentation and architecture diagrams	Low
PB-26	EP-17	Meeting up with the Hospital’s focal person for Dataset	High
PB-27	EP-17	Implement EfficientNet-B0 PyTorch model for 14-class chest x-ray classification	High
PB-28	EP-17	Adapt client training loop to load and preprocess 224×224 x-ray images	High
PB-29	EP-17	Validate federated x-ray model reaches ≥95% of centralized baseline AUC	High

PB-30	EP-17	Update aggregation layer to handle EfficientNet-B0 state dict (5.3M parameters)	Medium

2.3 User Stories

Story ID	Backlog ID	User Story	Priority	Acceptance Criteria	Focal Person responsible for this task
US-01	PB-01	Hospital IT Admin, FL Cluster via YAML.	High	YAML applies without error, status field shows "Pending" → "Active"	Faisal Khalid
US-02	PB-02	System Admin, operator to automatically recover from pod crashes.	High	Kill a client pod mid-round.	Faisal Khalid
US-03	PB-03	MLOps Engineer, Client nodes containerized.	High	Docker image builds successfully.	Faisal Khalid
US-04	PB-04	Clinical Data Engineer, Configure local training epochs.	High	Epoch count configurable via ConfigMap.	Shah Wali
US-05	PB-05	Backend Developer, gRPC APIs for model download and gradient upload.	High	Proto definitions compile.	Wajeeh ur Rehman
US-06	PB-06	Security Engineer, want client gradients encrypted so the server cannot reconstruct individuals.	High	SecAgg+ encrypts gradient with pairwise masks	Shah Wali
US-07	PB-07	Security Engineer, want the aggregation server to compute the global update without accessing individual gradients.	High	Server runs SecAgg+ aggregation; only sum is reconstructed	Shah Wali
US-08	PB-08	MLOps, Kubeflow Pipeline DAG defining each.	High	DAG executes 10 rounds; each step logs	M. Hammad
US-09	PB-09	MLOps Engineer, per-round metrics in MLflow.	High	MLflow UI shows accuracy/loss per round.	M. Hammad
US-10	PB-10	System Admin, health checks on client pods so that unhealthy nodes are excluded.	High	Liveness probe fails on stuck pod	Faisal Khalid
US-11	PB-11	HealthCare Researcher, failed nodes automatically rescheduled.	High	Delete node pod; operator creates replacement.	Faisal Khalid
US-12	PB-12	HealthCare Researcher, synthetic patient data with hospital-specific.	Medium	4 datasets generated; feature distributions differ across nodes	Wajeeh ur Rehman

US-13	PB-13	HealthCare Researcher, Centralized training baseline.	Medium	Centralized model trains on merged data.	Wajeeh ur Rehman
US-14	PB-14	ML Ops, Communication overhead measured per round.	Medium	Bytes sent/received logged per client.	M. Hammad
US-15	PB-15	Software Engineer, JWT authentication on all APIs.	Medium	Unauthenticated requests are rejected.	Wajeeh ur Rehman
US-16	PB-16	Hospital IT, Different users see only permitted resources.	Medium	Engineer sees pipelines; Auditor sees logs only; unauthorized access blocked	Faisal Khalid
US-17	PB-17	System Admin, Structured logs for operator events.	Medium	Logs in JSON format	M. Hammad
US-18	PB-18	ML Ops, Prometheus metrics for round latency and alert on SLA breaches.	Medium	FL_round_duration_seconds exposed; alert fires if >30s	M. Hammad
US-19	PB-19	Hospital IT, Dashboard view of training progress	Medium	Kubeflow Dashboard accessible via browser	M. Hammad
US-20	PB-20	System Admin, Failed rounds	Medium	Round failure triggers email alert within 60s	M. Hammad
US-21	PB-21	System Admin, Deployment guide.	Medium	Guide contains step-by-step K3s setup	Faisal Khalid
US-22	PB-22	Security Engineer, TLS 1.3 encryption for control-plane traffic.	Medium	TLS handshake verified via packet capture.	Shah Wali
US-23	PB-23	ML Ops, Reproducible environment manifests	Low	requirements.txt pins versions	M. Hammad
US-24	PB-24	Developer, CI/CD pipelines so that code quality is enforced before merge.	Low	GitHub Actions runs pytest and ruff on PR	Wajeeh ur Rehman
US-25	PB-25	Healthcare Ai Researcher, want X-Ray Dataset to split across 4 nodes	High	Four dataset partitions are created; pathology label counts differ per node	Wajeeh ur Rehman
US-26	PB-26	ML Engineer, want an EfficientNet-B0 model for chest x-ray classification	High	Model loads ImageNet pretrained weights; state dict serializes without error	Shah Wali
US-27	PB-27	Clinical Data Engineer, want client training to process x-ray images instead of tabular data	High	Training loop loads .jpg files from local directory; batch training completes on 4-node cluster	Faisal Khalid
US-28	PB-28	Healthcare AI Researcher, want the federated x-ray model to match centralized performance	High	Federated training (4 nodes, 10 rounds) achieves AUC $\geq 0.95 \times$ centralized baseline on tested dataset	Shah Wali

US-29	PB-29	MLOps Engineer, want aggregation to handle CNN weight tensors from multiple clients	Medium	Server aggregates EfficientNet-B0 state dicts from four-clients; validation AUC improves after aggregation	Faisal Khalid

2.4 Story Points & Estimation

Story ID	User Story (Short)	Complexity Level	Story Points	Justification
US-01	Define FL Cluster YAML	Medium	5	Requires CRD schema design, validation, and kubectl integration
US-02	Auto-recovery from crashes	High	8	Complex reconciliation logic; pod watch streams; state machine handling
US-03	Containerize client nodes	Medium	3	Standard Docker packaging; PyTorch base image
US-04	Configurable local training	Low	3	ConfigMap injection; epoch loop parameterization
US-05	gRPC API definitions	Medium	5	Proto design; code generation; client/server stubs
US-06	Flower SecAgg+ client	High	13	Cryptographic protocol integration; Flower framework dependency setup
US-07	SecAgg+ aggregation server	High	13	SecAgg+ protocol server implementation; memory inspection testing
US-08	Kubeflow DAG	High	8	Pipeline SDK; component definitions; artifact passing
US-09	MLflow tracking	Medium	5	Experiment creation; metric logging; artifact storage
US-10	Health checks	Medium	3	Kubernetes probe definitions; endpoint exposure
US-11	Auto-reschedule nodes	High	8	Operator event handling; pod creation; state re-synchronization
US-12	Synthetic dataset gen	Medium	5	Statistical distribution modeling; CSV/Parquet output
US-13	Centralized baseline	Low	3	Standard PyTorch training; accuracy evaluation
US-14	Communication overhead	Low	2	Byte counting; logging wrapper
US-15	JWT authentication	Medium	5	Token generation; middleware; expiration handling
US-16	RBAC implementation	Medium	5	Role definitions; permission matrix; middleware enforcement
US-17	Structured logging	Low	2	JSON formatter; context injection

US-18	Prometheus metrics	Medium	3	Counter/gauge definitions; HTTP endpoint
US-19	Dashboard configuration	Low	2	Kubeflow UI setup; namespace config
US-20	Alerting integration	Medium	5	Webhook configuration; message templating
US-21	Deployment Guide	Medium	3	Documentation effort plus one clean-environment walkthrough
US-22	TLS 1.3 control Plane	Medium	5	MVP scope certificates, Kubernetes Secrets, handshake verification, rotation procedure
US-23	Environment manifest	Low	2	Pin versions in requirements.txt then separately reconcile the actual dependency-file convention; add Docker Compose manifest
US-24	CI/CD pipeline	Low	3	GitHub Actions running and ruff; scaffold already exists
US-25	X-Ray Dataset Partition	Medium	8	patient ID deduplication; stratified split by pathology labels; file I/O to 4 directories
US-26	EfficientNet-B0 model	High	13	PyTorch model definition; ImageNet weight loading; GPU compatibility testing
US-27	Image training pipeline	Medium	8	Image loader; data augmentation; batch preprocessing; integration with existing training loop
US-28	Federated validation	High	8	Centralized baseline training; federated 10-round execution; reproducibility verification
US-29	CNN aggregation	Medium	5	State dict shape handling; multi-layer tensor aggregation; model loading verification

2.5 Sprint Planning & Sprint Backlog

Sprint No.	Duration	Sprint Goal	Total Story Points
Sprint 1	2 Weeks	K3s cluster setup and scaffolding	US-03, US-23 = 5
Sprint 2	2 Weeks	CRD design, operator scaffolding & GenAI model Setup	US-01, US-17, US-26 = 7
Sprint 3	2 Weeks	Client runtime, local training & image data pipline	US-04, US-12, US-25, US-27 = 29
Sprint 4	2 Weeks	gRPC communication layer	US-05, US-10 = 8
Sprint 5	2 Weeks	Operator reconciliation and health monitoring	US-02, US-11 = 16
Sprint 6	2 Weeks	Flower SecAgg+ client integration	US-06 = 13
Sprint 7	2 Weeks	SecAgg+ aggregation server	US-07 = 13
Sprint 8	2 Weeks	Kubeflow Pipeline DAG design	US-08 = 8

Sprint 9	2 Weeks	MLflow experiment tracking	US-09, US-19 = 7
Sprint 10	2 Weeks	Authentication, RBAC, GenAI validation	US-15, US-16, US-28 = 18
Sprint 11	2 Weeks	Benchmarking and centralized baseline	US-13, US-14 = 5
Sprint 12	2 Weeks	Fault tolerance hardening	US-10 (refinement), US-20 = 5
Sprint 13	2 Weeks	TLS and security hardening	US-22 = 5
Sprint 14	2 Weeks	Observability and metrics	US-18, US-17 (refinement) = 3
Sprint 15	2 Weeks	End-to-end integration testing	US-02 (refinement), US-08 (refinement) = 0
Sprint 16	2 Weeks	Dashboard and alerting polish	US-19 (refinement), US-20 (refinement) = 0
Sprint 17	2 Weeks	Documentation and deployment guide	US-21, US-24 = 6
Sprint 18	2 Weeks	Third-party deployment validation	US-21 (verification) = 0
Sprint 19	2 Weeks	Performance tuning and benchmark finalization	US-13 (refinement), US-14 (refinement) = 0
Sprint 20	2 Weeks	Final documentation, demo preparation, and buffer	US-24 (refinement), buffer = 0

3. Functional Requirements

3.1 Use Case Diagram

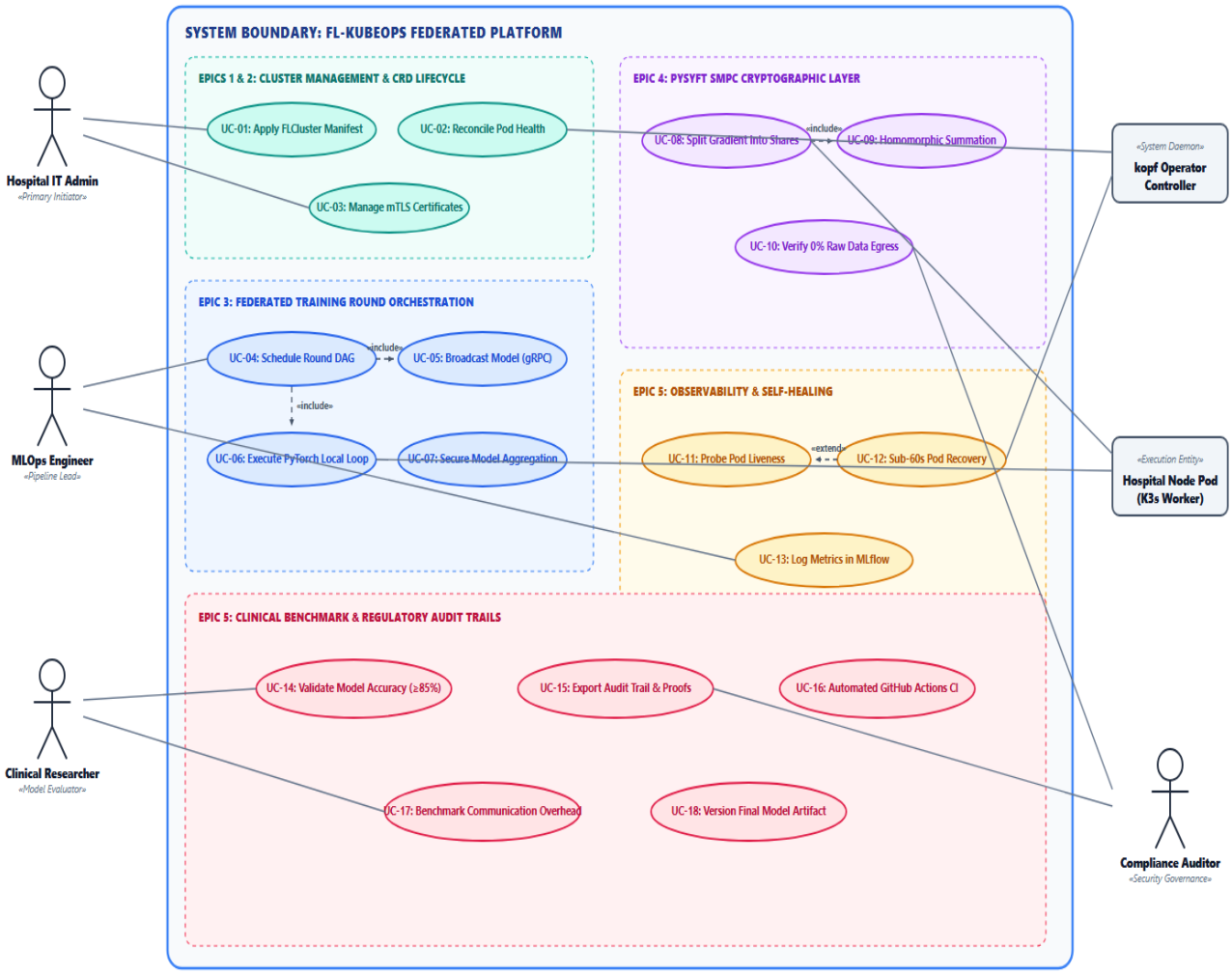


Figure8: Comprehensive UML Use Case Diagram

Use Case ID	Use Case Name	Primary Actor	Description	Mapped User Story ID	User Story
UC-01	Register FL Cluster	Hospital IT Admin	Creates a new federated learning cluster via YAML manifest	US-01	Define an FL Cluster via YAML.
UC-02	Monitor Cluster Health	Hospital IT Admin	Real-time status of all client nodes	US-02	Operator to automatically recover.
UC-03	Deploy Client Node	MLOps Engineer	Provisions a simulated hospital node.	US-03	I want client nodes containerized.
UC-04	Local Training	Clinical Data Engineer	Sets hyperparameters (epochs, batch size, learning rate) per node.	US-04	Configure local training epochs.
UC-05	Download Global Model	Hospital Node	Receives current global model weights.	US-05	gRPC API for model download.
UC-06	Perform Local Training	Hospital Node	Executes training loop.	US-04	Configure local training epochs.
UC-07	Compute Gradient Update	Hospital Node	Calculates weight deltas.	US-06	Client gradients split into secret shares.

UC-08	Encrypt Gradient (SecAgg+)	Hospital Node	Encrypts gradient using pairwise masking protocol	US-06	Client gradients encrypt, the server cannot reconstruct individual updates.
UC-09	Upload Encrypted Gradient	Hospital Node	SMPC shares to aggregation server	US-05	gRPC APIs for model download and gradient upload
UC-10	Aggregate Securely	MLOps Engineer	Shares compute to global update.	US-07	Aggregation server to compute the global update
UC-11	Update Global Model	MLOps Engineer	Aggregated update to global model	US-08	Kubeflow Pipeline DAG
UC-12	Validate Model	MLOps Engineer	Global model on hold-out validation set	US-09	Per-round metrics in MLflow
UC-13	Log Round Metrics	MLOps Engineer	Records accuracy, loss, to MLflow	US-09	Per-round metrics in MLflow
UC-14	Schedule Next Round	MLOps Engineer	Subsequent round via Kubeflow Pipeline DAG	US-08	Kubeflow Pipeline DAG
UC-15	Detect Node Failure	MLOps Engineer	Identifies unresponsive client pods	US-10	Health checks on client pods
UC-16	Recover Failed Node	MLOps Engineer	Reschedules failed pod	US-11	Failed nodes automatically rescheduled
UC-17	Authenticate Client	MLOps Engineer	Validates JWT token	US-15	JWT authentication on all APIs
UC-18	Enforce RBAC	MLOps Engineer	User role permissions for requested operations	US-16	RBAC so that different users see only permitted resources
UC-19	Audit Round Execution	Compliance Auditor	Generates immutable audit trail	US-16	RBAC
UC-20	Generate Compliance Report	Compliance Auditor	Exports PDF report of data-locality	US-16	RBAC
UC-21	View Dashboard	Hospital Governance Board	Kubeflow Dashboard to view pipeline status	US-19	Dashboard view
UC-22	Receive Failure Alert	System Administrator	Email notification for failure	US-20	Alerts for failed rounds
UC-23	Version Model Artifact	MLOps Engineer	Tags and stores model checkpoints	US-09	Per-round metrics in MLflow
UC-24	Run Benchmark Comparison	Healthcare AI Researcher	Executes centralized baseline	US-13	Centralized training baseline
UC-25	Measure Bandwidth	MLOps Engineer	Calculates bytes transmitted per client traffic	US-14	Communication overhead measured
UC-26	Deploy on K3s	Third-Party Evaluator	Follows published guide to deploy local K3s cluster	US-21	Deployment guide
UC-27	Trigger CI/CD Build	GitHub/CI System	Runs automated tests and linting on pull requests	US-24	CI/CD pipelines
UC-28	Rotate TLS Certificates	Security Engineer	Updates control-plane certificates before expiry	US-22	TLS 1.3 encryption
UC-29	Train X-Ray Classifier	Healthcare AI Researcher	Executes chest x-ray classification training	US-25,26,27,28	Dataset partitioned; EfficientNet-B0 trained using federated learning.

4. Non-functional Requirements

4.1 Reliability

- **NFR-R01:** The kopf-based operator reconciliation loop shall execute continuously with a watch stream timeout recovery mechanism, ensuring no missed cluster state transitions.
- **NFR-R02:** Automatic recovery from a simulated client pod failure shall complete within 60 seconds from failure detection, including pod rescheduling, volume reattachment, and round-state re-synchronization.
- **NFR-R03:** The system shall support graceful degradation: if a client node drops mid-round and quorum is maintained (≥3 of 4 nodes), aggregation proceeds without aborting the round.
- **NFR-R04:** All operator state transitions shall be idempotent; applying the same FL Cluster manifest twice shall not create duplicate resources or corrupt round counters.

4.2 Performance

- **NFR-P01:** Aggregation latency per federated round (model broadcast → local training → SecAgg+ aggregation → validation) shall not exceed 30 seconds for 4 client nodes on CPU hardware (8 GB RAM, 4 cores) when training tabular models. For image classification (EfficientNet-B0 on 224×224 x-rays), per-round latency shall not exceed 120 seconds on GPU hardware (16 GB VRAM).
- **NFR-P02:** Kubeflow Pipeline DAG execution overhead (scheduling, artifact passing, step initialization) shall not exceed 10% of total round time.
- **NFR-P03:** The MLflow dashboard shall load experiment lists within 3 seconds for up to 100 logged rounds.
- **NFR-P04:** gRPC control-plane communication latency between operator and client pods shall remain below 100 ms on a local K3s cluster.

4.3 Use-ability

- **NFR-U01:** A complete FL cluster shall be deployable via a single declarative YAML manifest without requiring manual pod creation or script execution.
- **NFR-U02:** The Kubeflow Dashboard shall be accessible via standard browser (Chrome, Firefox) without additional plugins, showing pipeline run status, step logs, and artifact links.
- **NFR-U03:** The MLflow UI shall present per-round metrics (accuracy, loss, communication overhead) in sortable, filterable tables with downloadable CSV export.
- **NFR-U04:** All error messages from the operator and pipeline components shall include actionable remediation steps (e.g., "Client pod failed liveness probe: check ConfigMap dataset-path").

4.4 Other Non-functional Requirements

4.4.1 Security Requirements

- **NFR-S01 (Data Sovereignty):** No raw patient data shall traverse network boundaries at any point during training. Only cryptographically masked gradient shares (SMPC) may leave the client node.
- **NFR-S02 (SecAgg+ Irretrievability):** Flower SecAgg+ shall guarantee that individual client gradient contributions are cryptographically irretrievable from the aggregation server's view, verified by memory inspection during aggregation, dropout recovery tests (killing 1 of 4 clients mid-round)
- **NFR-S03 (Transport Security):** All control-plane communication (gRPC, Kubernetes API, MLflow tracking) shall use TLS 1.3 with certificates rotated every 90 days.

- **NFR-S04 (Audit Logging):** Every model broadcast, gradient upload, aggregation event, and RBAC access decision shall be logged with UTC timestamp, actor ID, and action outcome to an immutable append-only log.
- **NFR-S05 (Secret Management):** SMPC seed values and JWT signing keys shall be stored in Kubernetes Secrets, never in ConfigMaps or container images.

4.4.2 Scalability Requirements

- **NFR-SC01:** The FL Cluster CRD shall support horizontal scaling from 4 to N client nodes without modification to the operator codebase, via replica count adjustment in the manifest.
- **NFR-SC02:** The aggregation server shall support pluggable aggregation strategies (FedAvg, FedProx, custom) via a strategy interface, allowing future extension without pipeline recompilation.

4.4.3 Explain ability & Validation Requirements

- **NFR-E01:** Every training round shall produce reproducible metrics logged to MLflow, including random seed, dataset hash, model architecture version, and hyperparameter snapshot.
- **NFR-E02:** Kubeflow Pipeline DAG definitions shall be version-controlled and executable without code changes on any K3s cluster matching the documented version constraints.
- **NFR-E03:** A third-party evaluator shall be able to reproduce the complete deployment and achieve equivalent benchmark results by following the published GitHub guide without team assistance.

4.4.4 Compliance Requirements

- **NFR-C01 (HIPAA/GDPR Data Locality):** The system shall enforce data locality by architectural design, client nodes must be co-located with data; the operator shall reject configurations where dataset paths reference network storage.
- **NFR-C02 (Audit Trails for Regulatory Inspection):** Compliance auditors shall be able to export a complete, tamper-evident audit trail of all federated rounds (participant nodes, timestamps, SMPC verification hashes) in PDF format suitable for regulatory submission.

5. Scrum Activities Plan

The team tracks progress in Notion: sprint, story and task databases, a task board, and a burndown chart of story points remaining. Velocity is the completed story points per sprint, and it is reviewed at each retrospective to calibrate future commitments against the team's demonstrated capacity.

5.1 Kanban Board / Task Tracking

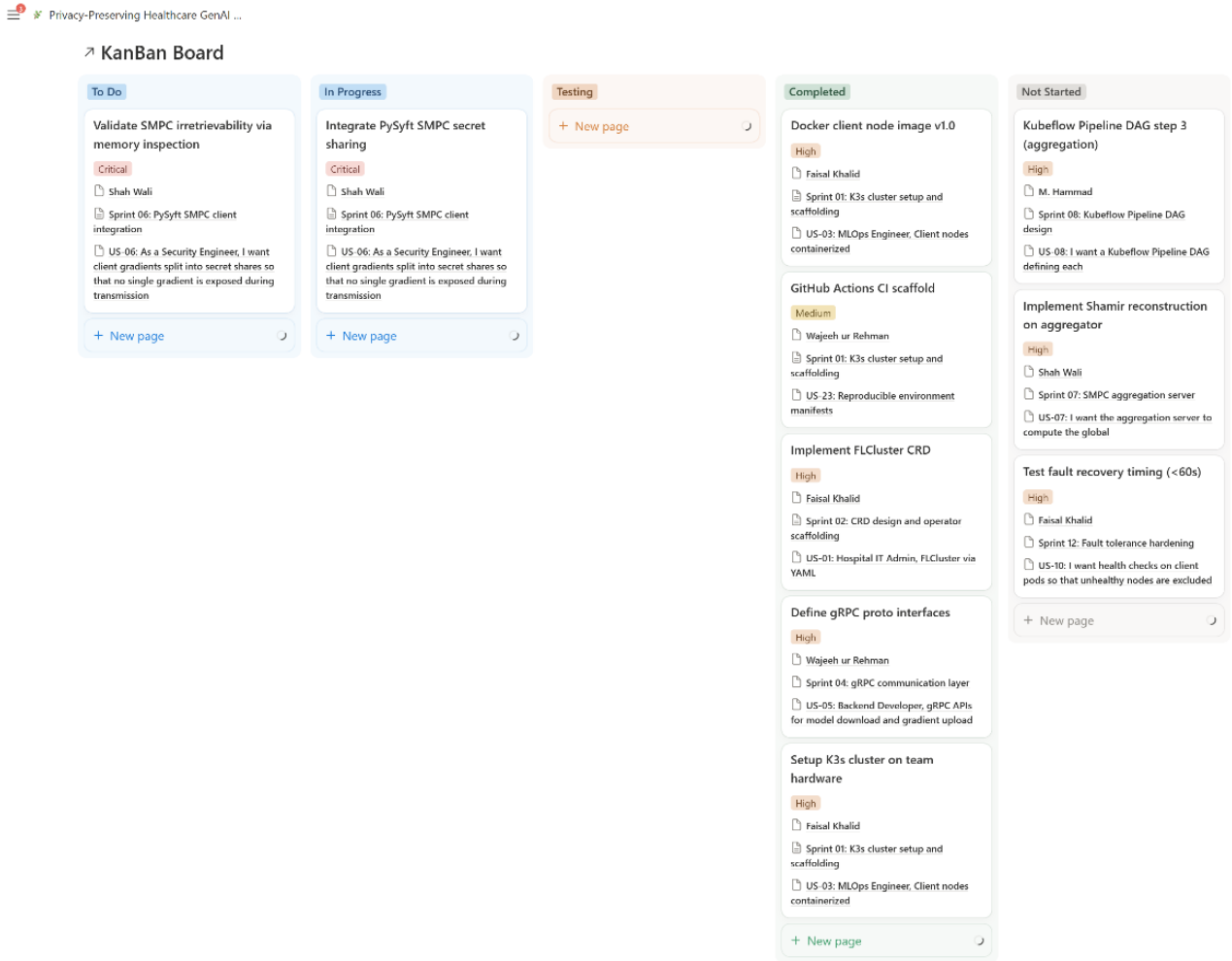


Figure9: FL-KubeOps Notion Sprint Board

Kanban Board

To Do	In Progress	Testing	Done
Validate SMPC irretrievability via memory inspection	Integrate PySyft SMPC secret sharing	—	Docker client node image v1.0
—	—	—	GitHub Actions CI scaffold
—	—	—	Implement FL Cluster CRD
—	—	—	Define gRPC proto interfaces
—	—	—	Setup K3s cluster on team hardware

5.2 Unit Testing

Framework: We use pytest for every test. GitHub Actions is configured to run pytest and flake8 on each PR. A failing test blocks the merge.

Isolation: Mocks and stubs replace external dependencies. Examples: Kubernetes API, gRPC channels, MLflow server. No test needs a running K3s cluster. unittest.mock.

Out of scope: Some behaviour needs a running system. Sections 5.3 and 5.4 verify it. Examples: 60-second recovery, SMPC memory inspection, TLS.

Test sequence: Setup, execute, verify, teardown. Setup builds fixtures, mocks, and a seed. Teardown resets mocks and deletes temporary files.

Process: We write the plan entry before coding. The story's focal person writes the tests. Tests ship in the same pull request. Unit names are placeholders until implementation.

Unit Test Plan Table

Unit / Function Under Test	Linked User Story	Testing Framework	Test Data	Test Sequence	Test Oracle
FL Cluster spec validation	US-01 (PB-01)	pytest	Valid: 4 clients, FedAvg, local dataset paths. Invalid: missing client's field. Invalid: unsupported strategy name. Invalid: negative client count. Invalid: network storage path (NFR-C01). Edge: exactly 4 clients; FedProx.	Load manifest, call validator, compare.	Valid manifest is accepted as "Pending". Invalid manifest names the faulty field.
Reconcile idempotency	US-01, US-02 (NFR-R04)	pytest	Same valid spec applied twice.	Mock empty cluster, apply twice, count creates.	Second apply creates nothing new. Round counter stays unchanged.
Client pod failure handler	US-02, US-11 (PB-02, PB-11)	pytest	Valid: client-2 pod deleted in 4-client cluster. Invalid: pod outside the FL Cluster. Edge: two pods deleted together.	Mock 4 pods, send event, inspect calls.	One replacement per failed client pod. No action for unrelated pods.
Quorum check	US-02 (NFR-R03)	pytest	Clients responding: 4, 3, 2, and 0.	Call check per count, compare result.	True for 4 and 3. False for 2 and 0.
Client health check	US-10 (PB-10)	pytest	Valid: training loop reports activity. Invalid: training loop stops reporting. Edge: check before first epoch.	Stub training loop, call check, compare status.	Healthy for active and pre-epoch states. Failure for the stalled state.
Training loop and parameter parsing	US-04 (PB-04)	pytest	Valid: epochs 1 and 3; batch 32. Invalid: epochs 0, -1, or non-numeric. Edge: batch size above dataset size.	Build tiny dataset, run loop, compare weights.	Loop runs exactly the configured epochs.
Synthetic dataset generator	US-12 (PB-12)	pytest	Valid: seed 42, 4 nodes. Invalid: 0 nodes. Edge: 1 node.	Generate twice with same seed; once different.	4 datasets are produced. Invalid input raises an error.

Gradient secret sharing (client)	US-06 (PB-06)	pytest	Valid: known tensor, 2 and 3 shares. Invalid: empty or non-tensor input. Edge: all-zero or single-element tensor.	Seed, split with PySyft, recombine shares.	At least 2 shares are produced. No single share equals the original.
Share aggregation (server)	US-07 (PB-07)	pytest	Valid: shares from 4 clients. Invalid: mismatched tensor shapes. Edge: shares from 3 of 4 clients.	Build shares, aggregate, compare with FedAvg.	Aggregate matches the expected FedAvg result. Mismatched shapes raise an error.
gRPC messages and servicer	US-05 (PB-05)	pytest	Valid: download and upload messages. Invalid: missing round identifier. Edge: empty payload.	Round-trip each message, call servicer.	Round-trip messages equal originals.
JWT validation	US-15 (PB-15)	pytest	Valid: unexpired token, test key. Invalid: missing, malformed, wrong signature, expired. Edge: token expiring this second.	Generate tokens with a test-only key.	Valid token returns its claims. Invalid tokens are rejected.
RBAC permission check	US-16 (PB-16)	pytest	Roles: Admin, Engineer, Auditor. Resources: pipelines, logs, admin operations. Invalid: unknown or empty role.	Call check for every pair, compare matrix.	Engineer may access pipelines. Auditor may access logs only. All other pairs are denied.
JSON log formatter	US-17 (PB-17), NFR-S04	pytest	Valid: event with actor and outcome. Edge: quotes, newline, non-ASCII text.	Format event, parse output as JSON.	Output is valid JSON. It has UTC timestamp, actor ID, outcome.
Communication overhead counter	US-14 (PB-14)	pytest	Payloads: 0 bytes, 1 KB, 1 MB. Four clients.	Pass payloads through wrapper, read logs.	Logged bytes equal payload sizes.
Round duration metric	US-18 (PB-18), NFR-P01	pytest	Durations: 12 s, 30 s, 31 s.	Record duration, read metric, call check.	Metric holds the recorded value. SLA flag is raised above 30 s.

5.3 Acceptance Criteria

Feature	Acceptance Conditions	Test Scenarios	Expected Results
SMPC Irretrievability	Individual gradients must not be re-constructible.	1. Run 4-client round with SMPC enabled. 2. Dump server process memory. 3. Inspect aggregation server logs.	No plaintext gradient tensors found; only masked shares present.
Fault Recovery Timing	System must recover from client pod failure within 60 seconds.	1. Start round 5 with 4 nodes. 2. Delete client-2 pod at epoch 3. 3. Measure time to restored quorum.	Recovery logged within 60s; round completes with ≥3 nodes.
Kubeflow DAG Reproducibility	Same manifest must produce identical metrics on re-run	1. Execute pipeline with seed=42. 2. Re-execute with identical manifest. 3. Compare MLflow metrics.	Accuracy and loss curves match within ±0.1%.
Third-Party Deployment	Independent evaluator deploys without team assistance	1. Team member not involved in docs follows GitHub README. 2. Deploys on fresh K3s VM. 3. Runs 10-round benchmark.	Deployment succeeds; benchmark achieves ≥85% centralized accuracy.
Declarative Single-Manifest	One YAML applies entire cluster	1. Run kubectl apply -f FL Cluster.yaml. 2. Verify pods, services, configmaps created. 3. Check operator logs.	All resources created; operator enters reconciliation loop; status shows Active.

5.4 Release Criteria

Release Gate	Completeness Criterion	Validation Method	Pass Threshold
RG-01	Kubernetes Operator Implementation	kubectl get crd FL Clusters.fl.kubeops.io returns CRD; operator pod runs without crash loops; status reflects real-time.	100% CRD lifecycle coverage
RG-02	FL Training Round Execution	Automated end-to-end test executes broadcast → training → aggregation → validation across 4 nodes	Zero manual steps; all 4 nodes complete
RG-03	Secure Aggregation (PySyft)	Memory inspection + formal protocol test; intermediate server data dumped and searched for plaintext gradients	No individual gradient recoverable
RG-04	Fault Tolerance and Recovery	Chaos test: random pod deletion mid-round; recovery time logged	≤60 seconds from failure to quorum restore
RG-05	Kubeflow Pipeline Integration	Pipeline SDK compiles DAG; executes 10 rounds; MLflow shows all runs	10 rounds logged; all artifacts versioned

RG-06	Benchmarking and KPI Verification	Federated vs. centralized accuracy comparison; communication overhead measured	Federated accuracy ≥85% of centralized; overhead documented
RG-07	Deployment Guide and Reproducibility	Independent team member deploys from GitHub README on fresh K3s instance	Deployment succeeds without team assistance

5.5 Burndown Chart & Progress Tracking

Our team tracks progress via **Notion sprint boards** and the built-in **burndown chart gadget**.

Sprint Tracking table (Ideal vs Actual):

Sprint	Planned Points	Completed Points	Remaining Points	Status
Sprint 1	5	5	114	Completed
Sprint 2	7	7	107	Completed
Sprint 3	8	8	99	Completed
Sprint 4	8	8	91	Completed
Sprint 5	16	16	75	Completed
Sprint 6	13	0	62	Active
Sprint 7	13	0	49	Upcoming
Sprint 8	8	0	41	Upcoming
Sprint 9	7	0	34	Upcoming
Sprint 10	10	0	24	Upcoming
Sprint 11	5	0	19	Upcoming
Sprint 12	5	0	14	Upcoming
Sprint 13	5	0	9	Upcoming
Sprint 14	3	0	6	Upcoming
Sprint 15	0	0	6	Upcoming
Sprint 16	0	0	6	Upcoming
Sprint 17	6	0	0	Upcoming
Sprint 18	0	0	0	Upcoming
Sprint 19	0	0	0	Upcoming
Sprint 20	0	0	0	Upcoming

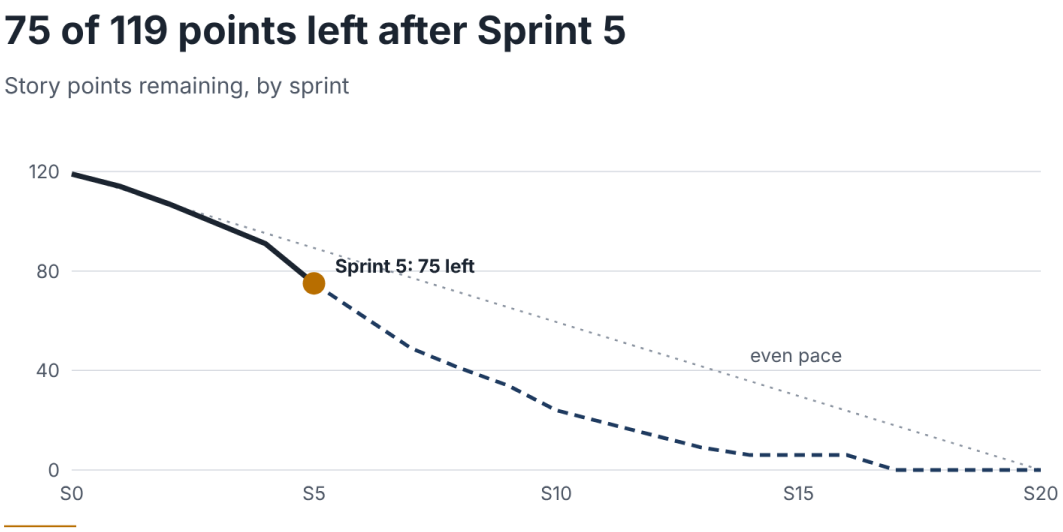


Figure10: FL-KubeOps 20-Sprint Burndown Path

5.6 Backlog Refinement Plan

Frequency: Once per sprint (every 2 weeks), held mid-sprint (Wednesday of the second week)

Process:

- **Review K3s cluster behavior:** Check for operator crashes, resource exhaustion on commodity laptops, and PySyft compatibility issues.
- **Assess PySyft compatibility:** Verify SMPC primitives work with the Python version pinned in the Docker images; update if upstream breaking changes occur.
- **Advisor feedback integration:** Prof Dr. Saira Andleeb Gillani reviews sprint demos and suggests reprioritization; high-priority feedback creates new backlog items for the next sprint.

Definition of ready update: Ensure all backlog items entering the next sprint have acceptance criteria, story point estimates, and assigned focal persons.

5.7 Version Control

We use Git with GitHub (Section 2, Tools) to manage the FL-KubeOps code, to track each member's contributions, and to link development work to sprint user stories. All work lives in one shared repository: https://github.com/fl-kubeops.

Items under version control: the kopf operator code, the FL Cluster CRD manifest, the client Dockerfile and training scripts, the gRPC proto files, the Kubeflow Pipeline DAG definitions (NFR-E02), requirements.txt and Docker Compose files with pinned versions (R-01, R-08), the K3s deployment guide (US-21), the Sphinx documentation, and the GitHub Actions workflows. SMPC

seed values and JWT signing keys are never committed; they are stored only in Kubernetes Secrets (NFR-S05).

Workflow:

US-02 Operator recovery from client pod failure → feature/US-02-pod-recovery → **Commit → Pull Request → Review/Test → Merge to main → Done**

Conventions:

- **Branching:** main is the only long-lived branch. Each user story gets one branch created from main, named **feature/US-<ID>-<short-description>**. For Bug fixes, use **fix/US-<ID>-<short-description>**.
- **Commits:** small, single-purpose changes with a type prefix and the story ID, for example "feat: US-02 handle client pod deletion in reconcile loop". The types we use are feat, fix, test, docs, and chore.
- **Pull requests:** one per story, titled with the story ID. The description lists the Backlog ID, the acceptance criteria from Section 2.3, and how the change was tested.
- **Review and merge:** a team member other than the author reviews and runs the change. The pytest and flake8 checks should pass before merge; automated enforcement is delivered by US-24. Approved branches merge into main and are then deleted.
- **Traceability:** each Notion story card links to its pull request, and each pull request names its story, so the board, the burndown figures in Section 5.5, and the Git history can be cross-checked.

Sprint tags: at the end of each sprint we tag main as **sprint-<N>** (for example **sprint-05**), so the code state for any sprint can be recovered and matched to the Revision History & Sprint Log.

Sprint 5			
User Story ID	US-02	**Backlog ID**	PB-02
Assignee	Faisal Khalid	**Priority**	High
Activity	**What We Do**		**Example (US-02)**
Repository	Use one shared project repository		https://github.com/fl-kubeops/operator
Branch	Created one branch per user story		feature/US-02-pod-recovery
Commit	Commit small, meaningful changes with the story ID		feat: US-02 handle client pod deletion in reconcile loop
Pull Request	Submit the completed story for review		US-02: Operator recovery from client pod failure.
Review & Test	M Hammad, runs pytest, and runs the Section 5.3 scenario for the story		Kill a client pod mid-round; reviewer confirms the operator recovers it
Merge	Merge approved code into main		Pull request → main
Contribution	Keep each member's work visible in the history		Commits and pull requests by each member, taken from the GitHub history

Contribution Tracking

Team Member	Role	Stories (focal person, Section 2.3)
Faisal Khalid	KubeOps lead	US-01, US-02, US-03, US-10, US-11, US-16, US-21
Shah Wali	FL security lead	US-04, US-06, US-07, US-22
Wajeeh ur Rehman	Pipeline/MLOps	US-05, US-12, US-13, US-15, US-24
M. Hammad	Backend integration	US-08, US-09, US-14, US-17, US-18, US-19, US-20, US-23

6. Risk Analysis

Risk ID	Description	Impact	Mitigation Strategy
R-01	PySyft SMPC compatibility issues with latest Python/Kubernetes versions	High	Pin PySyft and Python versions in Docker images; maintain a compatibility matrix.
R-02	K3s resource constraints on team hardware (commodity laptops)	Medium	Limit client nodes to 4; use CPU-only PyTorch.
R-03	Limited availability of federated healthcare benchmark datasets in Pakistan	Medium	Generate high-fidelity synthetic patient data using Bayesian network models conditioned on NHANES feature distributions.
R-04	Complexity of kopf operator reconciliation loops	High	Start with a minimal reconciliation loop.
R-05	Regulatory ambiguity in Pakistan's health data policy	Medium	Design system to exceed current draft requirements.
R-06	Team skill gap in Kubernetes operator patterns	Medium	kopf tutorial series in Sprint 0; pair programming fallback to Helm charts
R-07	Mid-round node failure during demo/evaluation	High	Implement deterministic checkpointing every 2 rounds.
R-08	Kubeflow Pipelines SDK breaking changes	Medium	Pin SDK version in requirements.txt; subscribe to Kubeflow release notes.
R-09	Scope creep: adding too many aggregation algorithms instead of focusing on FedAvg + SMPC	High	Strict sprint gate: no new aggregation strategies until CC-3 (SMPC) and CC-6 (Benchmarking) are complete.
R-10	Third-party deployment failure during reproducibility testing	Medium	Write deployment guide incrementally (Sprint 17); test with a team member who did not write the guide (Sprint 18).
R-11	Risk of exceeding the 60-second recovery limit	Medium	Run at least 10 recovery trials

7. References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS), Fort Lauderdale, FL, 2017, pp. 1273–1282.
[2] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS), Dallas, TX, 2017, pp. 1175–1191.
[3] M. Abadi et al., "Deep learning with differential privacy," in Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS), Vienna, Austria, 2016, pp. 308–318.
[4] T. Li, A. K. Diao, V. Smith, Y. Papailiopoulos, and V. Smola, "Federated learning: Challenges, methods, and future directions," IEEE Signal Process. Mag., vol. 37, no. 3, pp. 50–60, May 2020.
[5] N. Rieke et al., "The future of digital health with federated learning," NPJ Digit. Med., vol. 3, no. 1, pp. 1–7, 2020.
[6] D. J. Beutel et al., "Flower: A friendly federated learning research framework," arXiv preprint arXiv:2007.14390, 2020.
[7] G. A. Reina et al., "OpenFL: An open-source framework for federated learning," arXiv preprint arXiv:2105.06413, 2021.
[8] T. Li, A. Hu, A. Babakniya, C. Ma, and M. Sanjabi, "FedProx: Tackling heterogeneity in federated learning," arXiv preprint arXiv:1812.06127, 2020.
[9] The Kubeflow Authors, Kubeflow: Machine Learning Toolkit for Kubernetes, v1.8, 2024. [Online]. Available: https://www.kubeflow.org/docs/
[10] Zalando SE, kopf: Kubernetes Operator Pythonic Framework, v1.37, 2024. [Online]. Available: https://kopf.readthedocs.io/
[11] The Linux Foundation, K3s: Lightweight Kubernetes, 2024. [Online]. Available: https://k3s.io/
[12] MLflow Authors, MLflow: An Open Source Platform for the Machine Learning Lifecycle, v2.x, 2024. [Online]. Available: https://mlflow.org/
[13] OpenMined, PySyft: A system for privacy-preserving AI, 2024. [Online]. Available: https://github.com/OpenMined/PySyft
[14] J. Dobies and J. Wood, Kubernetes Operators: Automating the Container Orchestration Platform, O'Reilly Media, 2021.
[15] L. Sweeney, "Simple demographics often identify people uniquely," Carnegie Mellon University, Data Privacy Working Paper 3, Pittsburgh, PA, 2000.
[16] Grand View Research, Privacy Enhancing Technologies Market Report, 2024-2030, 2024.
[17] Precedence Research, Privacy Preserving AI Market Size to Hit USD 39.93 Billion by 2035, 2026.
[18] Future Market Insights, Federated Learning Healthcare Market Analysis, 2026
[19] Beutel, D. J., Topal, T., Mathur, A., Qiu, X., Parcollet, T., & Lane, N. D. (2020). Flower: A Friendly Federated Learning Framework. arXiv preprint arXiv:2007.14390.
[20] Irvin, J., Rajpurkar, P., Ko, M., Yu, Y., Ciurea-Ilcus, S., Chute, C., ... & Ng, A. Y. (2019). CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison. Proceedings of the AAAI Conference on Artificial Intelligence, 33(01), 590-597.
[21] Xu, G., Li, H., Ren, H., Yang, K., Deng, R. H., & Cheng, X. (2022). Secure Aggregation for Federated Learning in Flower. Proceedings of the First International Workshop on Federated Learning for User Privacy and Data Confidentiality (ICML 2022 Workshop).

Appendix A: Glossary

Term	Definition
AUC (Area Under the ROC Curve)	Performance metric for classification models. AUC = 1.0 is perfect discrimination; AUC = 0.5 is random guessing. Standard metric for medical diagnostic models
Shaukat Khanam Dataset	Dataset provided by Shaukat khanam for FYP basis only
EfficientNet-B0	Convolutional neural network architecture for image classification. 5.3 million parameters
Flower	Open-source federated learning framework (Apache-2.0 license). Provides SecAgg+ secure aggregation and client/server runtime abstractions.
Federated Learning (FL)	A distributed machine learning paradigm where models are trained across decentralized devices holding local data samples, without exchanging raw data.
SecAgg+ (Secure Aggregation Plus)	Cryptographic protocol from Bonawitz et al. (2017) for federated aggregation. The server learns only the sum of client updates, never individual contributions. Implemented by Flower framework (Salvia). Tolerates up to 1 client dropping mid-round (3-of-4 threshold). Assumes semi-honest threat model where participants follow the protocol but may attempt to infer others' data.
Kubernetes Operator	A method of packaging, deploying, and managing a Kubernetes application using custom resources and controller logic.
Custom Resource Definition (CRD)	A Kubernetes API extension that defines a new object type (e.g., FL Cluster) stored in the etcd database.
Kopf	A Python framework for building Kubernetes operators using decorators and asynchronous event handling.
Kubeflow Pipeline	A platform for building and deploying portable, scalable machine learning workflows based on Docker containers and Kubernetes.
K3s	A lightweight, certified Kubernetes distribution designed for resource-constrained environments and edge computing.
MLflow	An open-source platform for managing the end-to-end machine learning lifecycle, including experiment tracking and model versioning.
PySyft	An open-source framework by OpenMined for privacy-preserving AI, providing SMPC, differential privacy, and federated learning primitives.
FL Cluster	The custom resource defined by FL-KubeOps representing a federated learning deployment (clients, server, strategy, and security parameters).
Gradient Aggregation	The process of combining model updates (gradients) from multiple clients into a single global update.
Data Sovereignty	The principle that data is subject to the laws and governance structures of the nation or facility where it is physically located.
Differential Privacy	A mathematical framework for sharing information about a dataset by describing patterns of groups while withholding information about individuals.
Homomorphic Encryption	A form of encryption permitting computation on ciphertexts, generating encrypted results that decrypt to match plaintext computation results.
Re-identification Attack	A technique where anonymized datasets are linked with auxiliary data to re-identify specific individuals.
MLOps	A set of practices for collaboration and communication between data scientists and operations professionals to manage production ML lifecycles.
DAG (Directed Acyclic Graph)	A graph structure used in pipeline frameworks where nodes represent tasks and edges represent dependencies, with no cyclic dependencies.
Reconciliation Loop	The core control mechanism of a Kubernetes operator that continuously compares desired state (CRD spec) with observed state (cluster reality) and acts to converge them.

Appendix B: Sprint and Phase Retrospective Report

Advisor Name Signature

Retrospective Overview

A retrospective is held at the end of each two-week sprint and a longer one at the end of each project phase. Each entry records what was planned, what was finished, what carried over, and what the team will change. Sprint velocity from the Notion burndown (Section 5.5) is reviewed at every retrospective. The Scrum Master, Faisal Khalid, keeps the log, and the facilitator role rotates among the four team members.
The entry below covers Sprint 5, the most recent sprint recorded as Completed in Section 5.5 (Sprint 6 is Active).

Retrospective Header

Team Name / Project Name	*Group F26CS023, FL-KubeOps (Privacy-Preserving Healthcare GenAI)*
Phase	*Phase 1 SRS*
Sprint / Cycle Number	*Sprint 5 (sprint goal: Operator reconciliation and health monitoring)*
Date	*31/Oct/2026*
Facilitator	*Faisal Khalid*
Attendees	*Faisal Khalid, Shah Wali, Wajeeh ur Rehman, M. Hammad*
Supervisor Present	*Yes*

Sprint Summary: Planned, Achieved, and Carried Forward

Type	Item / Artefact ID	Description	Status	If Not Achieved, Why
Epic	EP-03	Server-Side Orchestration: kopf-based operator managing FL Cluster CRD lifecycle and round scheduling	Achieved	___
Epic	EP-07	Fault Tolerance & Recovery: automatic detection and recovery of client pod failures within 60 seconds	Partially Achieved	Recovery works.
User Story	US-02 (PB-02)	As a System Admin, I want the operator to recover automatically from client pod crashes. Acceptance: kill a client pod mid-round.	Achieved	___
User Story	US-11 (PB-11)	As a system operator, I want failed nodes rescheduled automatically. Acceptance: delete a node pod and the operator creates a replacement.	Achieved	___

Carried Forward to Next Sprint

Carried Forward = 0, no item carried forward.

Item / Artefact ID	Description	New Target Sprint	Notes
---	---	---	----

Follow-up on Previous Action Items

#	Action Item (from last retrospective)	Owner	Status	Notes
1	*Answer the Idea Defense objection on column deletion and encryption*	*Wajeeh ur rehamn/ Shah wali*	*Done*	*Addressed in v1.1, Section 1.1*

Discussion: Start, Stop, Continue

Category	Notes
Start doing	• *Record the acceptance-check result of each story in its pull request and Notion card. Run at least 10 recovery trials and record whether the timer starts at pod deletion or at detection.*
Stop doing	• *Stop letting the SRS, the sprint log and the Notion board drift apart; sprint points must follow the story estimates in Section 2.4.*
Continue doing	• *Continue tracking sprints in Notion and reviewing velocity from the burndown at each retrospective.* • *Continue two-week sprints with a mid-sprint refinement.*

New Action Items

Owners and due dates are proposed from the focal-person assignments in Section 2.3 and are to be confirmed by the team.

#	Action	Owner	Due Date	Linked To	Status
1	PySyft SMPC client integration	Shah Wali	*Next sprint*	*US-06*	*Open*

Metrics Snapshot

A quick, honest snapshot of the sprint's numbers. This does not need to be precise to the story point; it exists to spot trends across phases, such as consistently under-planning or the same task type repeatedly slipping.

#	Action	Owner	Due Date	Linked To	Status
1	Add an acceptance-check, field in pull request and Notion story cards	Hammad	*Sprint 6*	*Section 5.3, 5.7*	*Open*
2	Run at least 10 recovery trials	Faisal Khalid	*Sprint 6*	*US-02, US-11*	*Open*
3	Keep the SRS, sprint log, and Notion Board updated	Faisal Khalid	*Every Sprint*	*Section 2.5, 5.5*	*Open*
4	Pin PySyft and Python versions in the client and server Docker images	Wajeeh/ Hammad	*Sprint 6*	*US-06, R-01*	*Open*

University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

Faculty of Information Technology and Computer Science

Semester wise SDP Meeting Report

Group ID	Student Roll Number	Student Name and Signatures	Advisor
F26CS023	L1F23BSCS0192	Faisal Khalid	
	L1F23BSCS0199	Shah Wali	
	L1F23BSCS0669	Wajeeh Ur Rehman	
	L1F23BSCS0204	M. Hammad	

Sr.	Date	Status (P/A/L)				Agenda Items	Notes
		Student 1	Student 2	Student 3	Student 4		
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							

Date: Advisor's Signatures: _____